

Stat 145 (Multivariate Statistics)

Lesson 3.5 Cluster Analysis

Introduction

Large amounts of data are collected every day from satellite images, bio-medical, security, marketing, web search, geo-spatial or other automatic equipment. Mining knowledge from these big data far exceeds human's abilities.

Clustering is one of the important data mining methods for discovering knowledge in multidimensional data. The goal of clustering is to identify pattern or groups of similar objects within a data set of interest. In the literature, it is referred as “pattern recognition” or “unsupervised machine learning” - “unsupervised” because we are not guided by a priori ideas of which variables or samples belong in which clusters. “Learning” because the machine algorithm “learns” how to cluster.

Cluster analysis is popular in many fields, including:

- In cancer research for classifying patients into subgroups according their gene expression profile. This can be useful for identifying the molecular profile of patients with good or bad prognostic, as well as for understanding the disease.
- In marketing for market segmentation by identifying subgroups of customers with similar profiles and who might be receptive to a particular form of advertising.
- In City-planning for identifying groups of houses according to their type, value and location.

Clustering Distance Measures

The classification of observations into groups requires some methods for computing the distance or the (dis)similarity between each pair of observations. The result of this computation is known as a dissimilarity or distance matrix.

Methods for measuring distances

The choice of distance measures is a critical step in clustering. It defines how the similarity of two elements (x, y) is calculated and it will influence the shape of the clusters.

The classical methods for distance measures are Euclidean and Manhattan distances, which are defined as follow:

1. *Euclidean distance*

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. *Manhattan distance*

$$d = \sum_{i=1}^n |x_i - y_i|$$

Other dissimilarity measures exist such as correlation-based distances, which is widely used for gene expression data analyses. Correlation-based distance is defined by subtracting the correlation coefficient from 1. Different types of correlation methods can be used such as:

3. *Pearson correlation distance*

$$d = 1 - \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

4. *Eisen cosine correlation distance*

$$d = 1 - \frac{\left| \sum_{i=1}^n x_i y_i \right|}{\sqrt{\sum_{i=1}^n x_i^2 \sum_{i=1}^n y_i^2}}$$

3. *Spearman correlation distance:*

$$d = 1 - \frac{\sum_{i=1}^n (x'_i - \bar{x}')(y'_i - \bar{y}')}{\sqrt{\sum_{i=1}^n (x'_i - \bar{x}')^2 \sum_{i=1}^n (y'_i - \bar{y}')^2}}$$

where $x'_i = \text{rank}(x_i)$ and $y'_i = \text{rank}(y_i)$.

QUESTION: What type of distance measures should we choose?

The choice of distance measures is very important, as it has a strong influence on the clustering results. For most common clustering software, the default distance measure is the Euclidean distance.

Depending on the type of the data and the researcher questions, other dissimilarity measures might be preferred. For example, correlation-based distance is often used in gene expression data analysis.

Correlation-based distance considers two objects to be similar if their features are highly correlated, even though the observed values may be far apart in terms of Euclidean distance. The distance between two objects is 0 when they are perfectly correlated. Pearson’s correlation is quite sensitive to outliers. This does not matter when clustering samples, because the correlation is over thousands of genes. When clustering genes, it is important to be aware of the possible impact of outliers. This can be mitigated by using Spearman’s correlation instead of Pearson’s correlation.

If we want to identify clusters of observations with the same overall profiles regardless of their magnitudes, then we should go with correlation-based distance as a dissimilarity measure. This is particularly the case in gene expression data analysis, where we might want to consider genes similar when they are “up” and “down” together. It is also the case, in marketing if we want to identify group of shoppers with the same preference in term of items, regardless of the volume of items they bought. If Euclidean distance is chosen, then observations with high values of features will be clustered together. The same holds true for observations with low values of features.

Data Standardization

The value of distance measures is intimately related to the scale on which measurements are made. Therefore, variables are often scaled (i.e. standardized) before measuring the inter-observation dissimilarities. This is particularly recommended when variables are measured in different scales (e.g: kilograms, kilometers, centimeters, . . .); otherwise, the dissimilarity measures obtained will be severely affected.

The goal is to make the variables comparable. Generally variables are scaled to have i) standard deviation one and ii) mean zero.

The standardization of data is an approach widely used in the context of gene expression data analysis before clustering. We might also want to scale the data when the mean and/or the standard deviation of variables are largely different.

When scaling variables, the data can be transformed as follow:

$$\frac{x_i - center(x_i)}{scale(x_i)}$$

where $center(x)$ can be the mean or the median of x values, and $scale(x)$ can be the standard deviation (SD), the interquartile range, or the MAD (median absolute deviation).

The R base function `scale()` can be used to standardize the data. It takes a numeric matrix as an input and performs the scaling on the columns.

We’ll use the USArrests data as demo data sets. We’ll use only a subset of the data by taking 15 random rows among the 50 rows in the data set. This is done by using the function `sample()`. Next, we standardize the data using the function `scale()`:

```
set.seed(123)
ss <- sample(1:50, 15) #Take 15 random rows
df <- USArrests[ss, ] #Subset the 15 rows
df.scaled <- scale(df) #Standardize the variables
```

There are many R functions for computing distances between pairs of observations:

1. *dist()* R base function [stats package]: Accepts only numeric data as an input.
2. *get_dist()* function [factoextra package]: Accepts only numeric data as an input. Compared to the standard *dist()* function, it supports correlation-based distance measures including “pearson”, “kendall” and “spearman” methods.
3. *daisy()* function [cluster package]: Able to handle other variable types (e.g. nominal, ordinal, (a)symmetric binary). In that case, the Gower’s coefficient will be automatically used as the metric. It’s one of the most popular measures of proximity for mixed data types.

To compute Euclidean distance, you can use the R base *dist()* function, as follow:

```
dist.eucl <- dist(df.scaled, method = "euclidean")
```

Note that, allowed values for the option *method* include one of: “euclidean”, “maximum”, “manhattan”, “canberra”, “binary”, “minkowski”.

To make it easier to see the distance information generated by the *dist()* function, you can reformat the distance vector into a matrix using the *as.matrix()* function.

```
round(as.matrix(dist.eucl)[1:3, 1:3], 1)
```

```
##           New Mexico Iowa Indiana
## New Mexico      0.0  4.1     2.5
## Iowa            4.1  0.0     1.8
## Indiana         2.5  1.8     0.0
```

Correlation-based distances are commonly used in gene expression data analysis. The function *get_dist()* [factoextra package] can be used to compute correlation-based distances. Correlation method can be either “pearson”, “spearman” or “kendall”.

```
library("factoextra")
```

```
## Loading required package: ggplot2
```

```
## Welcome! Want to learn more? See two factoextra-related books at https://goo.gl/ve3WE
```

```
dist.cor <- get_dist(df.scaled, method = "pearson")
# Display a subset
round(as.matrix(dist.cor)[1:3, 1:3], 1)
```

```
##           New Mexico Iowa Indiana
## New Mexico      0.0  1.7    2.0
## Iowa            1.7  0.0    0.3
## Indiana         2.0  0.3    0.0
```

The function *daisy()* [cluster package] provides a solution (Gower's metric) for computing the distance matrix, in the situation where the data contain no-numeric columns.

The R code below applies the *daisy()* function on flower data which contains factor, ordered and numeric variables:

```
library(cluster)
# Load data
data(flower)
head(flower, 3)
```

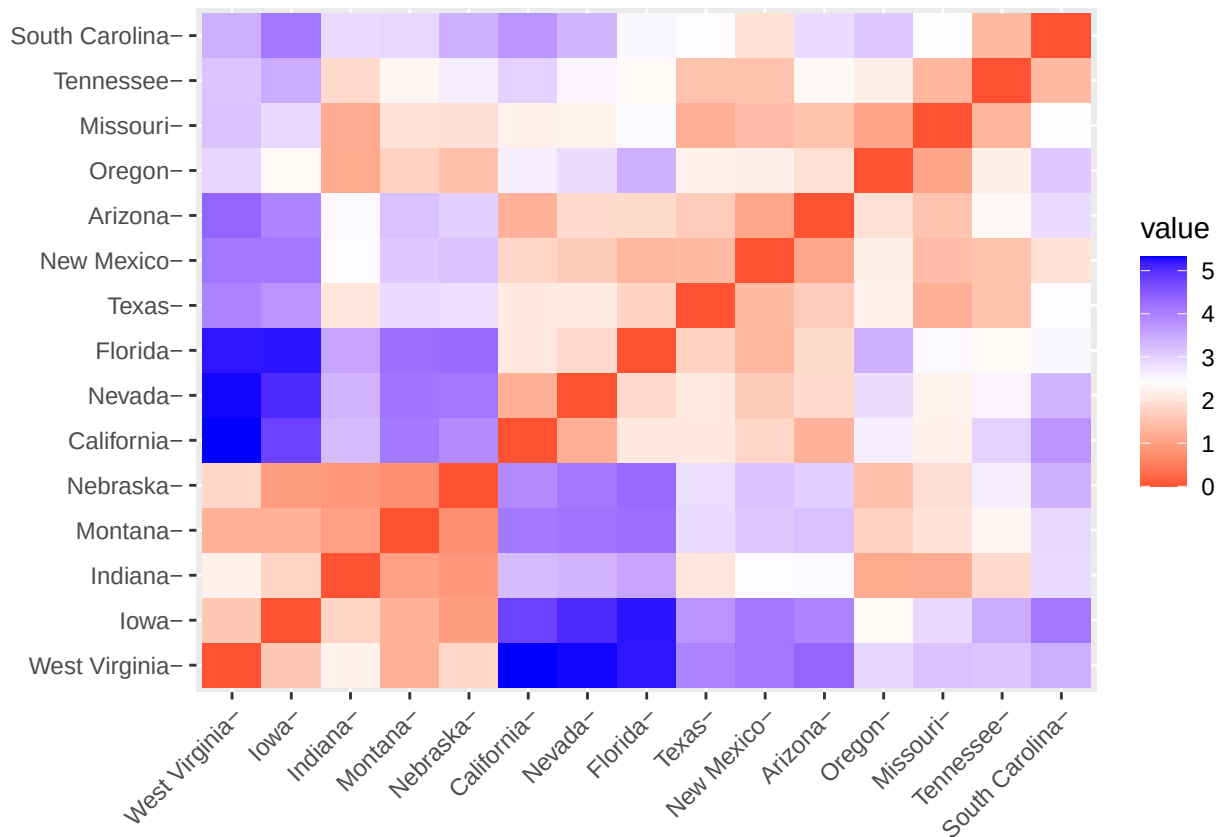
```
##   V1 V2 V3 V4 V5 V6  V7 V8
## 1  0  1  1  4  3 15  25 15
## 2  1  0  0  2  1  3 150 50
## 3  0  1  0  3  3  1 150 50
```

```
# Distance matrix
dd <- daisy(flower)
round(as.matrix(dd)[1:3, 1:3], 2)
```

```
##      1      2      3
## 1 0.00 0.89 0.53
## 2 0.89 0.00 0.51
## 3 0.53 0.51 0.00
```

A simple solution for visualizing the distance matrices is to use the function *fviz_dist()* [factoextra package].

```
library(factoextra)
fviz_dist(dist.eucl)
```



Partitioning Clustering

Partitioning clustering are clustering methods used to classify observations, within a data set, into multiple groups based on their similarity. The algorithms require the analyst to specify the number of clusters to be generated. We shall describe in this lesson the commonly used partitioning clustering, including:

- **K-means clustering** (MacQueen, 1967), in which, each cluster is represented by the center or means of the data points belonging to the cluster. The K-means method is sensitive to anomalous data points and outliers.
- **K-medoids clustering** or **PAM (Partitioning Around Medoids)** Kaufman & Rousseeuw (1990), in which, each cluster is represented by one of the objects in the cluster. PAM is less sensitive to outliers compared to k-means.
- **CLARA algorithm (Clustering Large Applications)**, which is an extension to PAM adapted for large data sets.

K-Means Clustering

K-means clustering is the most commonly used unsupervised machine learning algorithm for partitioning a given data set into a set of k groups (i.e. k clusters), where k represents the number of groups pre-specified by the analyst. It classifies objects in multiple groups (i.e., clusters), such that objects within the same cluster are as similar as possible (i.e., high intra-class similarity), whereas objects from different clusters are as dissimilar as possible (i.e., low inter-class similarity). In k-means clustering, each cluster is represented by its center (i.e, centroid) which corresponds to the mean of points assigned to the cluster.

The basic idea behind k-means clustering consists of defining clusters so that the total intra-cluster variation (known as total within-cluster variation) is minimized. There are several k-means algorithms available. The standard algorithm is the Hartigan-Wong algorithm (1979), which defines the total within-cluster variation as the sum of squared distances Euclidean distances between items and the corresponding centroid:

$$W(C_k) = \sum_{x_i \in C_k} (x_i - \mu_k)^2$$

- x_i design a data point belonging to the cluster C_k
- μ_k is the mean value of the points assigned to the cluster C_k

Each observation (x_i) is assigned to a given cluster such that the sum of squares (SS) distance of the observation to their assigned cluster centers μ_k is a minimum.

K-means algorithm can be summarized as follow:

1. Specify the number of clusters (K) to be created (by the analyst).
2. Select randomly k objects from the data set as the initial cluster centers or means.
3. Assigns each observation to their closest centroid, based on the Euclidean distance between the object and the centroid.
4. For each of the k clusters update the cluster centroid by calculating the new mean values of all the data points in the cluster. The centroid of a K th cluster is a vector of length p containing the means of all variables for the observations in the k th cluster; p is the number of variables.
5. Iteratively minimize the total within sum of square. That is, iterate steps 3 and 4 until the cluster assignments stop changing or the maximum number of iterations is reached.

By default, the R software uses 10 as the default value for the maximum number of iterations.

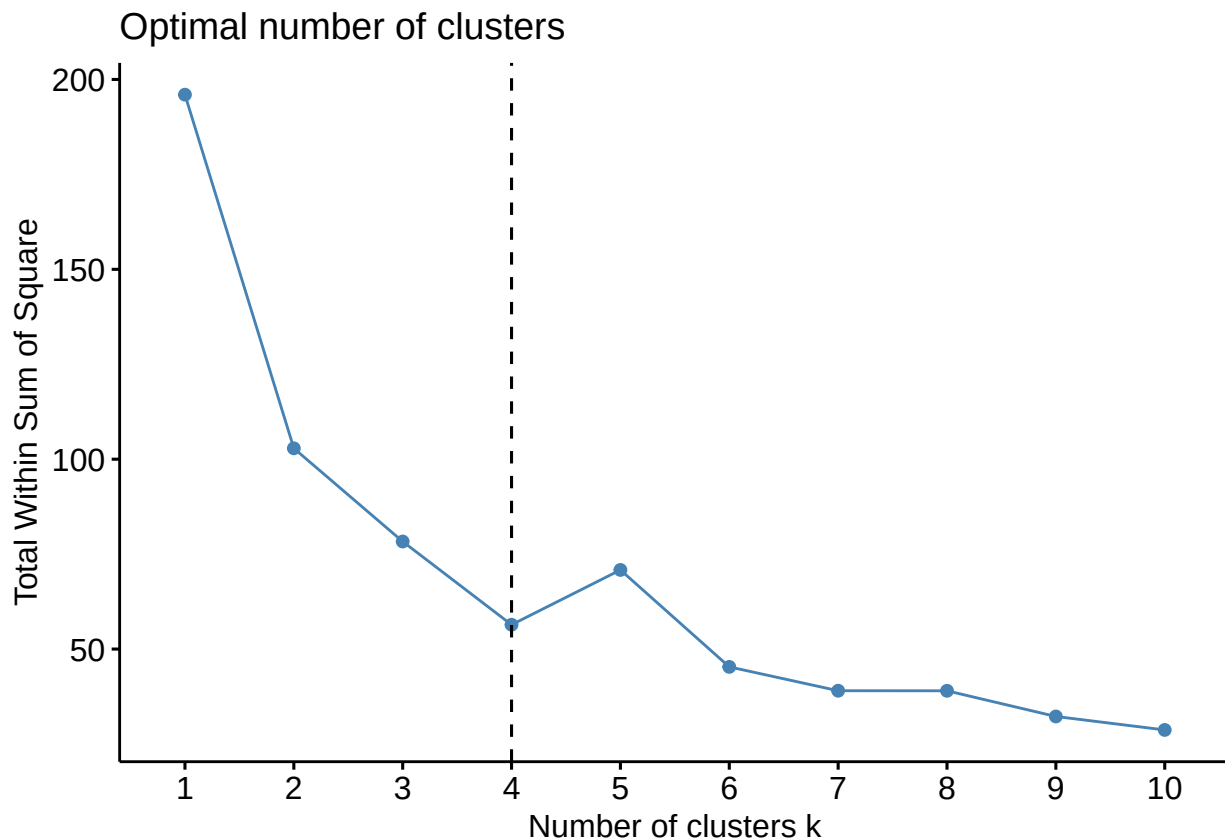
The standard R function for k-means clustering is `kmeans()` [stats package], which simplified format is as follow:

```
kmeans(x, centers, iter.max = 10, nstart = 1)
```

- x: numeric matrix, numeric data frame or a numeric vector
- centers: Possible values are the number of clusters (k) or a set of initial (distinct) cluster centers. If a number, a random set of (distinct) rows in x is chosen as the initial centers.
- iter.max: The maximum number of iterations allowed. Default value is 10.
- nstart: The number of random starting partitions when centers is a number.

The k-means clustering requires the users to specify the number of clusters to be generated. One simple solution is to compute k-means clustering using different values of clusters k. Next, the wss (within sum of square) is drawn according to the number of clusters. The location of a bend (knee) in the plot is generally considered as an indicator of the appropriate number of clusters.

```
library(factoextra)
df <- scale(USArrests)
fviz_nbclust(df, kmeans, method = "wss") +
  geom_vline(xintercept = 4, linetype = 2)
```



The plot above represents the variance within the clusters. It decreases as k increases, but it can be seen a bend (or “elbow”) at $k = 4$. This bend indicates that additional clusters beyond the fourth have little value.. In the next section, we’ll classify the observations into 4 clusters.

As k-means clustering algorithm starts with k randomly selected centroids, it’s always recommended to use the `set.seed()` function in order to set a seed for R’s random number generator.

```
#Compute k-means with k = 4
set.seed(123)
kmeans.out <- kmeans(df, 4, nstart = 25)
print(kmeans.out)
```

```
## K-means clustering with 4 clusters of sizes 8, 13, 16, 13
##
## Cluster means:
##      Murder      Assault      UrbanPop      Rape
## 1  1.4118898  0.8743346 -0.8145211  0.01927104
## 2 -0.9615407 -1.1066010 -0.9301069 -0.96676331
## 3 -0.4894375 -0.3826001  0.5758298 -0.26165379
## 4  0.6950701  1.0394414  0.7226370  1.27693964
##
## Clustering vector:
##      Alabama      Alaska      Arizona      Arkansas      California
##           1           4           4           1           4
##      Colorado      Connecticut      Delaware      Florida      Georgia
##           4           3           3           4           1
##      Hawaii      Idaho      Illinois      Indiana      Iowa
##           3           2           4           3           2
##      Kansas      Kentucky      Louisiana      Maine      Maryland
##           3           2           1           2           4
##      Massachusetts      Michigan      Minnesota      Mississippi      Missouri
##           3           4           2           1           4
##      Montana      Nebraska      Nevada      New Hampshire      New Jersey
##           2           2           4           2           3
##      New Mexico      New York      North Carolina      North Dakota      Ohio
##           4           4           1           2           3
##      Oklahoma      Oregon      Pennsylvania      Rhode Island      South Carolina
##           3           3           3           3           1
##      South Dakota      Tennessee      Texas      Utah      Vermont
##           2           1           4           3           2
##      Virginia      Washington      West Virginia      Wisconsin      Wyoming
##           3           3           2           2           3
##
```

```
## Within cluster sum of squares by cluster:
## [1]  8.316061 11.952463 16.212213 19.922437
## (between_SS / total_SS =  71.2 %)
##
## Available components:
##
## [1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
## [6] "betweenss"    "size"         "iter"         "ifault"
```

We can add the point classifications to the original data and compute the mean of each variables by clusters.

```
library(tidyverse)
df1 <- cbind(USArrests, cluster = kmeans.out$cluster)
head(df1)
```

```
##           Murder Assault UrbanPop Rape cluster
## Alabama      13.2      236       58 21.2      1
## Alaska       10.0      263       48 44.5      4
## Arizona       8.1      294       80 31.0      4
## Arkansas      8.8      190       50 19.5      1
## California    9.0      276       91 40.6      4
## Colorado      7.9      204       78 38.7      4
```

It is a good idea to plot the cluster results. These can be used to assess the choice of the number of clusters as well as comparing two different cluster analyses. Now, we want to visualize the data in a scatter plot with coloring each data point according to its cluster assignment.

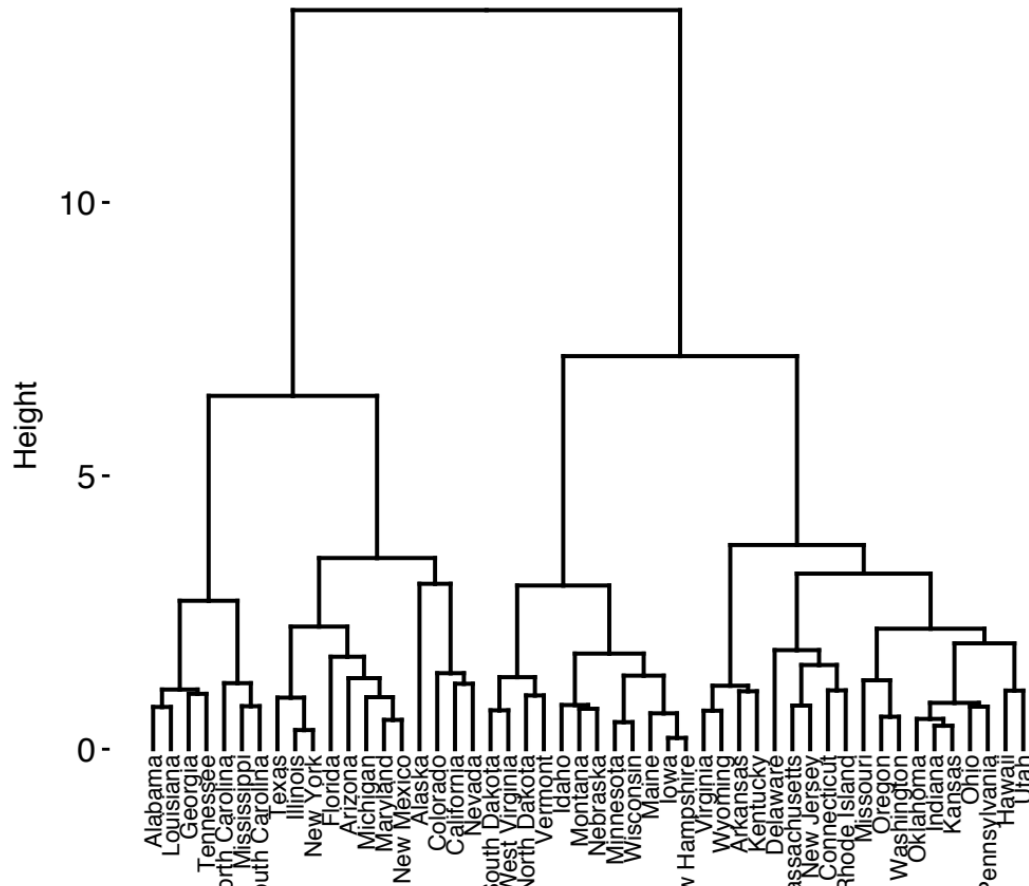
The function `fviz_cluster()` [factoextra package] can be used to easily visualize k-means clusters. It takes k-means results and the original data as arguments. In the resulting plot, observations are represented by points, using principal components if the number of variables is greater than 2. It's also possible to draw concentration ellipse around each cluster.

```
fviz_cluster(object = kmeans.out,
              data = df,
              palette = c("#2E9FDF", "#00AFBB", "#E7B800", "#FC4E07"),
              ellipse.type = "euclid", # Concentration ellipse
              star.plot = TRUE, # Add segments from centroids to items
              repel = TRUE, # Avoid label overplotting (slow)
              ggtheme = theme_minimal()
)
```

Hierarchical clustering [or hierarchical cluster analysis (HCA)] is an alternative approach to partitioning clustering (Part II) for grouping objects based on their similarity. In contrast to partitioning clustering, hierarchical clustering does not require to pre-specify the number of clusters to be produced.

- **Agglomerative clustering** in which, each observation is initially considered as a cluster of its own (leaf). Then, the most similar clusters are successively merged until there is just one single big cluster (root).
- **Divise clustering**, an inverse of agglomerative clustering, begins with the root, in witch all objects are included in one cluster. Then the most heterogeneous clusters are successively divided until all observation are in their own cluster.

11



Agglomerative Clustering

The **agglomerative clustering** is the most common type of hierarchical clustering use to group objects in clusters based on their similarity. It's also known as AGNES (Agglomerative Nesting). The algorithm starts by treating each object as a singleton cluster. Next, pairs of clusters are successively merged until all clusters have been merged into one big cluster containing all objects.

Agglomerative clustering works in a “bottom-up” manner. That is, each object is initially considered as a single-element cluster (leaf). At each step of the algorithm, the two clusters that are the most similar are combined into a new bigger cluster (nodes). This procedure is iterated until all points are member of just one single big cluster (root).

The inverse of agglomerative clustering is divisive clustering, which is also known as DIANA (Divise Analysis) and it works in a “top-down” manner. It begins with the root, in which all objects are included in a single cluster. At each step of iteration, the most heterogeneous cluster is divided into two. The process is iterated until all objects are in their own cluster.

Note that, agglomerative clustering is good at identifying small clusters. Divisive clustering is good at identifying large clusters.

The steps to agglomerative hierarchical clustering include:

1. Preparing the data.
2. Computing (dis)similarity information between every pair of objects in the data set.
3. Using linkage function to group objects into hierarchical cluster tree, based on the distance information generated at step 1. Objects/clusters that are in close proximity are linked together using the linkage function.
4. Determining where to cut the hierarchical tree into clusters. This creates a partition of the data.

Note that, it's generally recommended to standardize variables in the data set before performing subsequent analysis. Standardization makes variables comparable, when they are measured in different scales.

```
# Load the data
data("USArrests")
# Standardize the data
df <- scale(USArrests)
# Show the first 6 rows
head(df, nrow = 6)
```

```
##           Murder  Assault  UrbanPop      Rape
## Alabama    1.24256408 0.7828393 -0.5209066 -0.003416473
## Alaska     0.50786248 1.1068225 -1.2117642  2.484202941
## Arizona    0.07163341 1.4788032  0.9989801  1.042878388
## Arkansas   0.23234938 0.2308680 -1.0735927 -0.184916602
## California 0.27826823 1.2628144  1.7589234  2.067820292
## Colorado   0.02571456 0.3988593  0.8608085  1.864967207
```

```
# Compute the dissimilarity matrix
# df = the standardized data
res.dist <- dist(df, method = "euclidean")

as.matrix(res.dist)[1:6, 1:6]
```

```
##           Alabama  Alaska  Arizona  Arkansas  California  Colorado
## Alabama    0.000000 2.703754 2.293520 1.289810   3.263110 2.651067
## Alaska     2.703754 0.000000 2.700643 2.826039   3.012541 2.326519
## Arizona    2.293520 2.700643 0.000000 2.717758   1.310484 1.365031
## Arkansas   1.289810 2.826039 2.717758 0.000000   3.763641 2.831051
## California 3.263110 3.012541 1.310484 3.763641   0.000000 1.287619
## Colorado   2.651067 2.326519 1.365031 2.831051   1.287619 0.000000
```

The linkage function takes the distance information, returned by the function `dist()`, and groups pairs of objects into clusters based on their similarity. Next, these newly formed clusters are linked to each other to create bigger clusters. This process is iterated until all the objects in the original data set are linked together in a hierarchical tree.

For example, given a distance matrix “res.dist” generated by the function `dist()`, the R base function `hclust()` can be used to create the hierarchical tree.

```
res.hc <- hclust(d = res.dist, method = "ward.D2")
```

- *d*: a dissimilarity structure as produced by the `dist()` function.
- *method*: The agglomeration (linkage) method to be used for computing distance between clusters. Allowed values is one of “ward.D”, “ward.D2”, “single”, “complete”, “average”, “mcquitty”, “median” or “centroid”.

There are many cluster agglomeration methods (i.e, linkage methods). The most common linkage methods are described below.

- **Maximum or complete linkage**: The distance between two clusters is defined as the maximum value of all pairwise distances between the elements in cluster 1 and the elements in cluster 2. It tends to produce more compact clusters.
- **Minimum or single linkage**: The distance between two clusters is defined as the minimum value of all pairwise distances between the elements in cluster 1 and the elements in cluster 2. It tends to produce long, “loose” clusters.
- **Mean or average linkage**: The distance between two clusters is defined as the average distance between the elements in cluster 1 and the elements in cluster 2.
- **Centroid linkage**: The distance between two clusters is defined as the distance between the centroid for cluster 1 (a mean vector of length p variables) and the centroid for cluster 2.
- **Ward’s minimum variance method**: It minimizes the total within-cluster variance. At each step the pair of clusters with minimum between-cluster distance are merged.

Note that, at each stage of the clustering process the two clusters, that have the smallest linkage distance, are linked together. Complete linkage and Ward’s method are generally preferred.

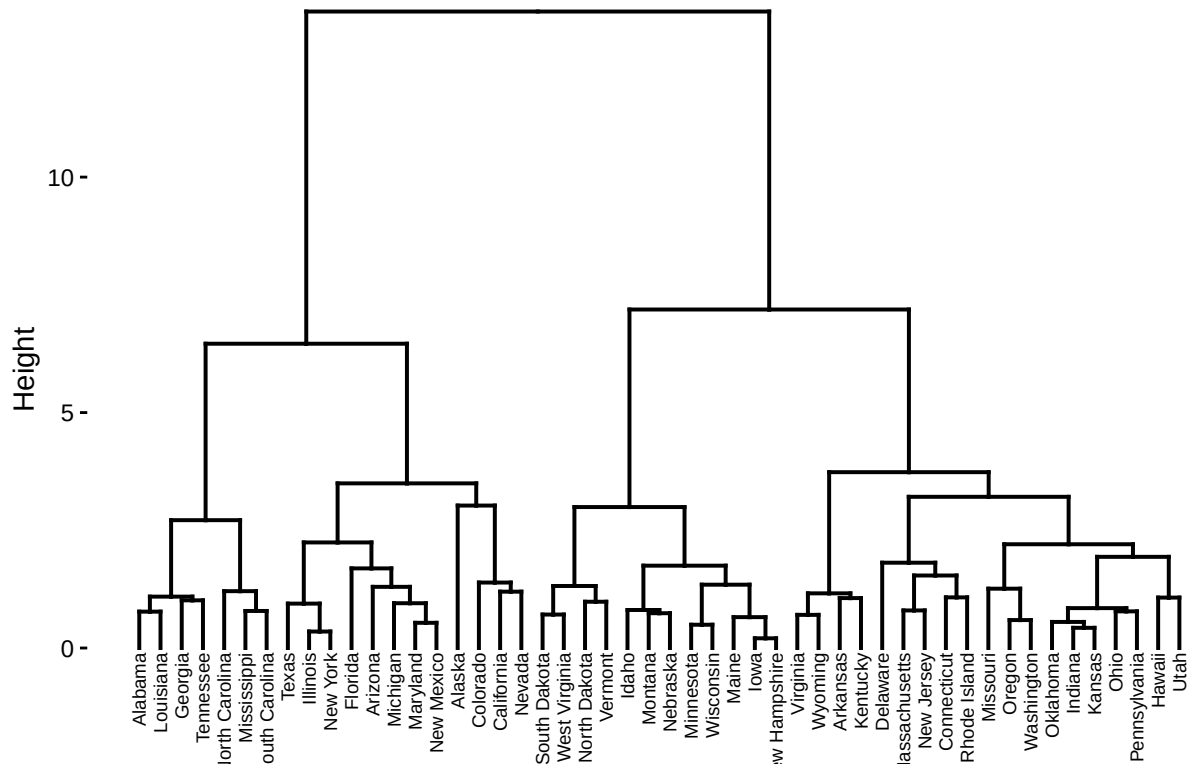
We visualize the results of the clustering by creating a dendrogram. Dendrograms correspond to the graphical representation of the hierarchical tree generated by the function `hclust()`.

```
res.hc <- hclust(d = res.dist, method = "ward.D2")
fviz_dend(res.hc, cex = 0.5)
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## i The deprecated feature was likely used in the factoextra package.
## Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

## Warning: The `<scale>` argument of `guides()` cannot be `FALSE`. Use "none" instead a
## of ggplot2 3.3.4.
## i The deprecated feature was likely used in the factoextra package.
## Please report the issue at <https://github.com/kassambara/factoextra/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

Cluster Dendrogram



In the dendrogram displayed above, each leaf corresponds to one object. As we move up the tree, objects that are similar to each other are combined into branches, which are themselves

fused at a higher height. The height of the fusion, provided on the vertical axis, indicates the (dis)similarity/distance between two objects/clusters. The higher the height of the fusion, the less similar the objects are. This height is known as the **cophenetic distance** between the two objects.

After linking the objects in a data set into a hierarchical cluster tree, you might want to assess that the distances (i.e., heights) in the tree reflect the original distances accurately. One way to measure how well the cluster tree generated by the *hclust()* function reflects your data is to compute the correlation between the cophenetic distances and the original distance data generated by the *dist()* function. If the clustering is valid, the linking of objects in the cluster tree should have a strong correlation with the distances between objects in the original distance matrix.

The closer the value of the correlation coefficient is to 1, the more accurately the clustering solution reflects your data. Values above 0.75 are felt to be good. The “average” linkage method appears to produce high values of this statistic. This may be one reason that it is so popular.

```
# Compute cophentic distance
res.coph <- cophenetic(res.hc)
# Correlation between cophenetic distance and
# the original distance
cor(res.dist, res.coph)
```

```
## [1] 0.6975266
```

Execute the *hclust()* function again using the average linkage method. Next, call *cophenetic()* to evaluate the clustering solution.

```
res.hc2 <- hclust(res.dist, method = "average")
cor(res.dist, cophenetic(res.hc2))
```

```
## [1] 0.7180382
```

The correlation coefficient shows that using a different linkage method creates a tree that represents the original distances slightly better.

Cut the dendrogram into different groups

One of the problems with hierarchical clustering is that, it does not tell us how many clusters there are, or where to cut the dendrogram to form clusters. You can cut the hierarchical tree at a given height in order to partition your data into clusters. The R base function *cutree()* can be used to cut a tree, generated by the *hclust()* function, into several groups either by specifying the desired number of groups or the cut height. It returns a vector containing the cluster number of each observation.

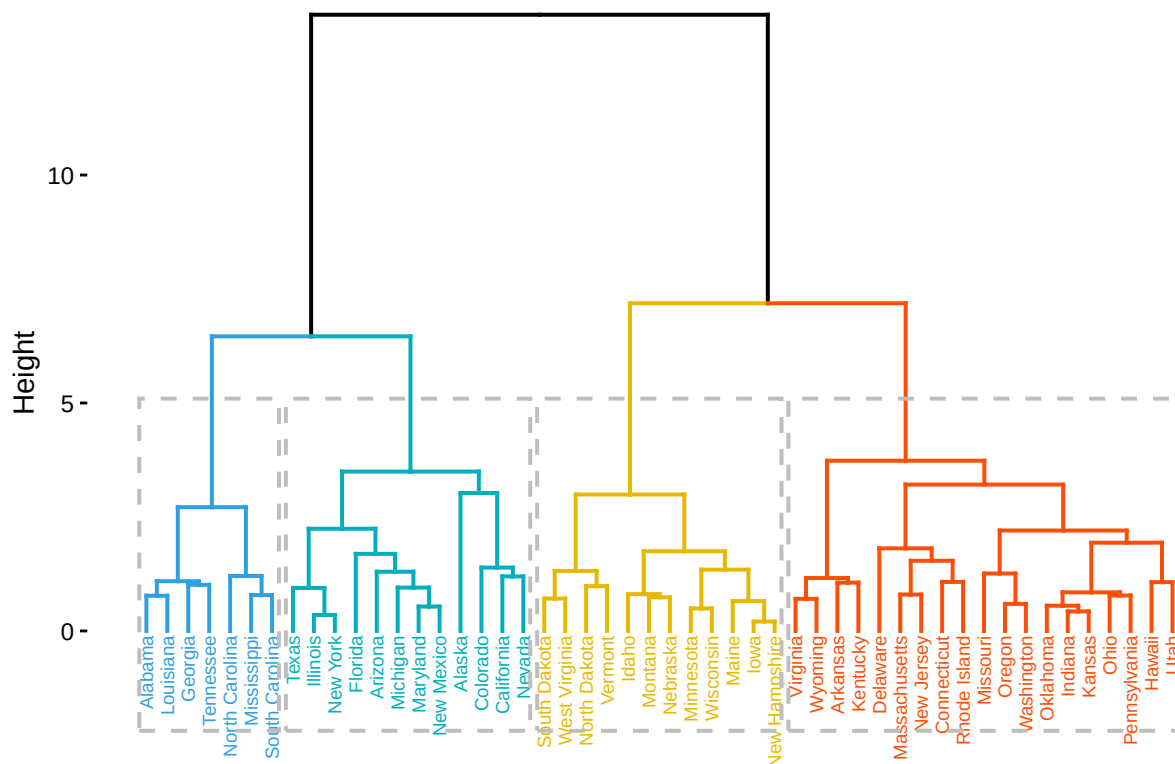

```
# Cut tree into 4 groups
grp <- cutree(res.hc, k = 4)
table(grp)
```

```
## grp
## 1  2  3  4
## 7 12 19 12
```

The result of the cuts can be visualized easily using the function `fviz_dend()` [in factoextra]:

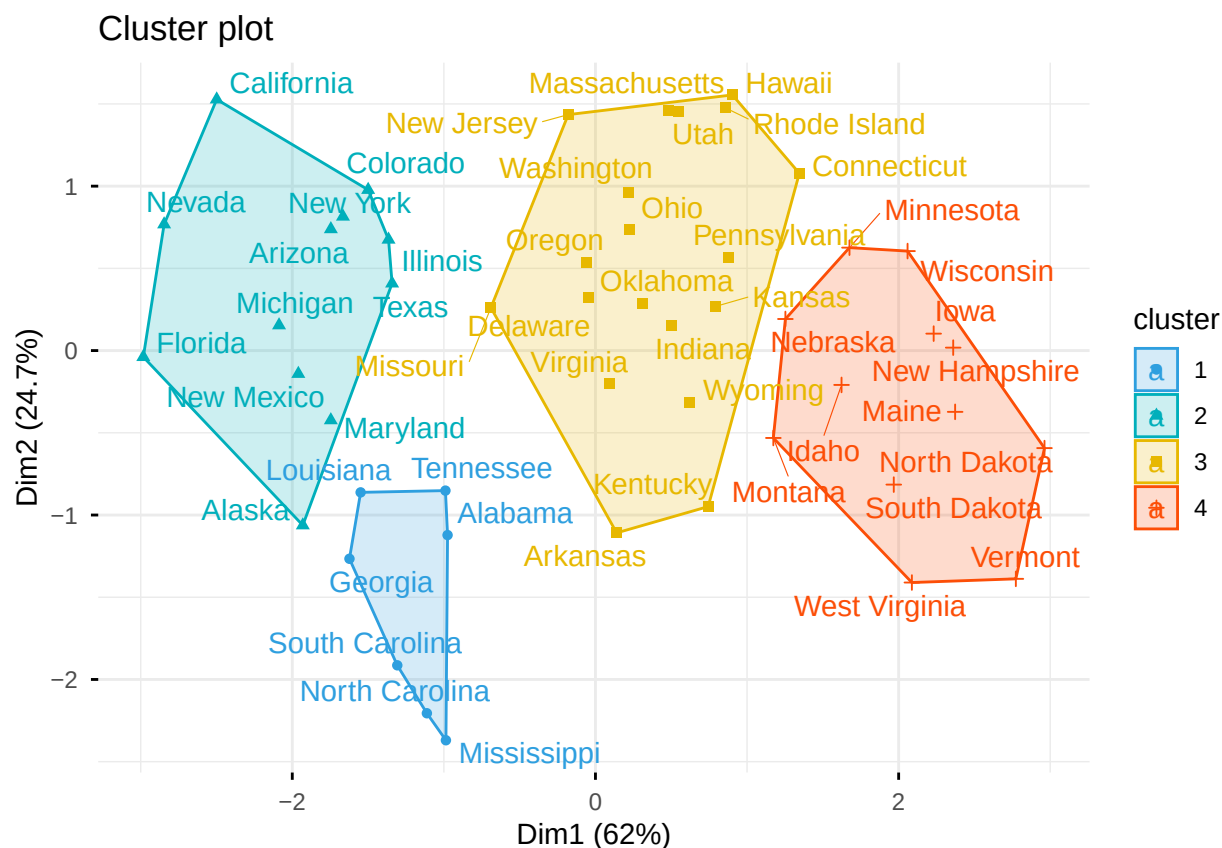
```
#Cut in 4 groups and color by groups
fviz_dend(res.hc, k = 4, # Cut in four groups
  cex = 0.5, # label size
  k_colors = c("#2E9FDF", "#00AFBB", "#E7B800", "#FC4E07"),
  color_labels_by_k = TRUE, # color labels by groups
  rect = TRUE # Add rectangle around groups
)
```

Cluster Dendrogram



Using the function `fviz_cluster()` [in factoextra], we can also visualize the result in a scatter plot. Observations are represented by points in the plot, using principal components. A frame is drawn around each cluster.

```
fviz_cluster(list(data = df, cluster = grp),
  palette = c("#2E9FDF", "#00AFBB", "#E7B800", "#FC4E07"),
  ellipse.type = "convex", # Concentration ellipse
  repel = TRUE, # Avoid label overplotting (slow)
  show.clust.cent = FALSE, ggtheme = theme_minimal()
)
```



The R package **cluster** makes it easy to perform cluster analysis in R. It provides the function *agnes()* and *diana()* for computing agglomerative and divisive clustering, respectively. These functions perform all the necessary steps for you. You don't need to execute the *scale()*, *dist()* and *hclust()* function separately.

The functions can be executed as follow:

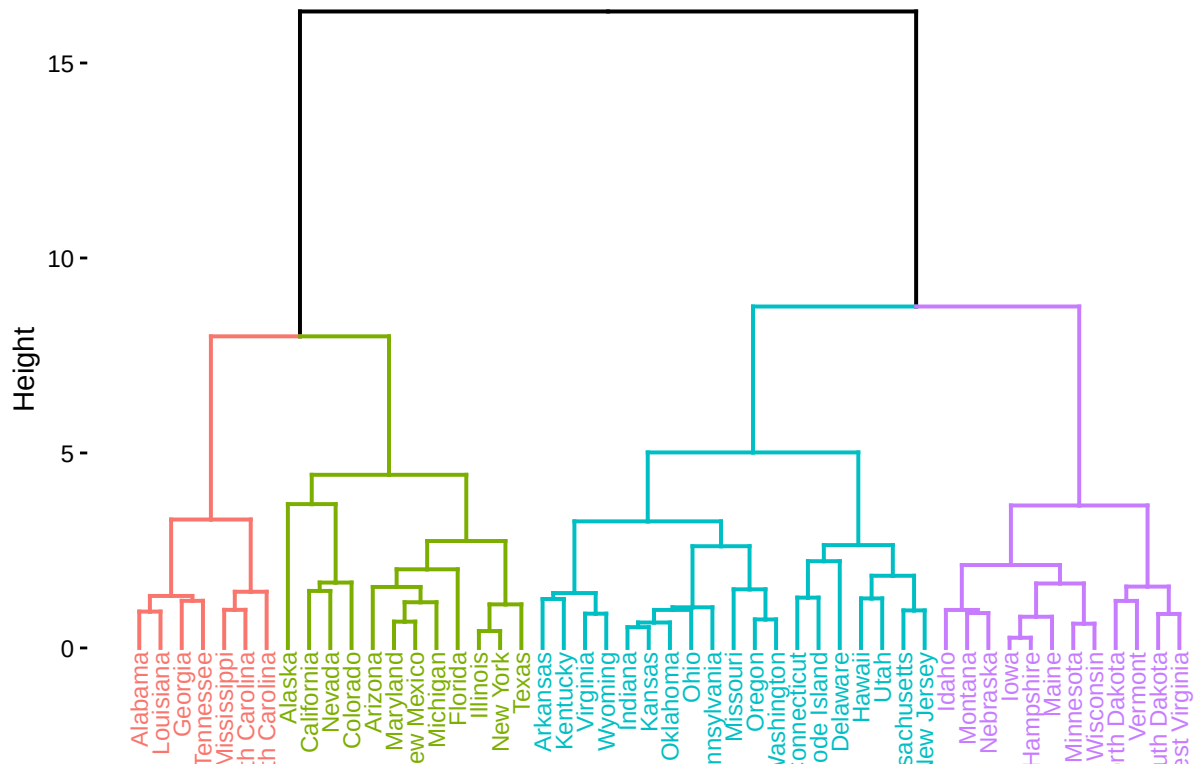
```
library("cluster")
# Agglomerative Nesting (Hierarchical Clustering)
res.agnes <- agnes(x = USArrests, # data matrix
  stand = TRUE, # Standardize the data
  metric = "euclidean", # metric for distance matrix
  method = "ward" # Linkage method
)
```

```
# Divisive ANALysis Clustering
res.diana <- diana(x = USArrests, # data matrix
                  stand = TRUE, # standardize the data
                  metric = "euclidean" # metric for distance matrix
                  )
```

After running `agnes()` and `diana()`, you can use the function `fviz_dend()` [in `factoextra`] to visualize the output:

```
fviz_dend(res.agnes, cex = 0.6, k = 4)
```

Cluster Dendrogram



Next we determine the desired number of clusters (e.g., 4) and cut the tree using `cutree()`. The `agnes()` object needs to be converted to an `hclust()` object first.

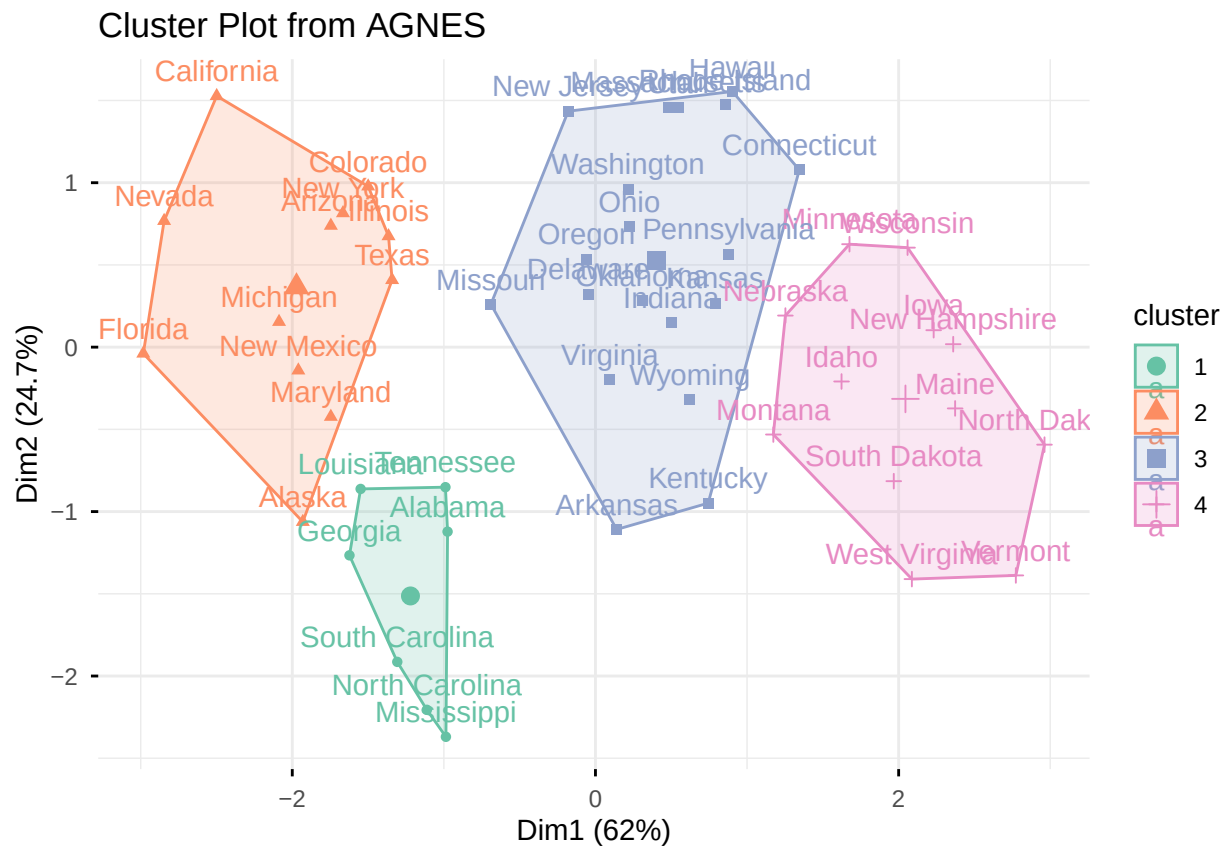
```
# Cut tree into 4 groups
grp1 <- cutree(as.hclust(res.agnes), k = 4)

# View the number of members in each cluster
table(grp1)
```

```
## grp1
```

```
## 1 2 3 4
## 7 12 19 12
```

```
fviz_cluster(list(data = df, cluster = grp1),
  ellipse.type = "convex", # Draws a convex hull around clusters
  palette = "Set2",        # Use a color palette
  ggtheme = theme_minimal(),
  main = "Cluster Plot from AGNES"
)
```



You explore other layout for displaying dendrograms.

```
library(igraph)
```

```
## Warning: package 'igraph' was built under R version 4.5.2
```

```
##
```

```
## Attaching package: 'igraph'
```

```
## The following objects are masked from 'package:lubridate':
```

```
##
```

```
## %--%, union
```

```
## The following objects are masked from 'package:dplyr':
##
##   as_data_frame, groups, union

## The following objects are masked from 'package:purrr':
##
##   compose, simplify

## The following object is masked from 'package:tidyr':
##
##   crossing

## The following object is masked from 'package:tibble':
##
##   as_data_frame

## The following objects are masked from 'package:stats':
##
##   decompose, spectrum

## The following object is masked from 'package:base':
##
##   union
```

```
fviz_dend(res.hc,
          k = 4,
          k_colors = "jco",
          type = "phylogenetic",
          repel = TRUE)
```

